

【GiantPandaCV 导语】自 NCNN 开源以来，其它厂商的端侧推理框架或者搭载特定硬件芯片的工具链层出不穷。如何去繁从简的阅读一个深度学习推理框架十分重要，这篇文章记录了我是如何阅读 NCNN 框架的，希望对一些不知道如何下手的读者有一点启发。

0x00. 想法来源

CNN 从 15 年的 ResNet 在 ImageNet 比赛中大放异彩，到今天各种层出不穷的网络结构被提出以解决生活中碰到的各种问题。然而，在 CNN 长期发展过程中，也伴随着很多的挑战，比如如何调整算法使得在特定场景或者说数据集上取得最好的精度，如何将学术界出色的算法落地到工业界，如何设计出在边缘端或者有限硬件条件下的定制化 CNN 等。前两天看到腾讯优图的文章：[腾讯优图开源这三年](#)，里面提到了 NCNN 背后的故事，十分感动和佩服，然后我也是白嫖了很多 NCNN 的算法实现以及一些调优技巧。所以为了让很多不太了解 NCNN 的人能更好的理解腾讯优图这个 "从 0 到 1" 的深度学习框架，我将结合我自己擅长的东西来介绍我眼中的 NCNN 它是什么样的？

0x01. 如何使用 NCNN

这篇文章的重点不是如何跑起来 NCNN 的各种 Demo，也不是如何使用 NCNN 来部署自己的业务网络，这部分没有什么比官方 wiki 介绍得更加清楚的资料了。所以这部分我只是简要汇总一些资料，以及说明一些我认为非常重要的东西。

官方 wiki 指路：<https://github.com/Tencent/ncnn/wiki>

在 NCNN 中新建一个自定义层教程：

https://github.com/Ewenwan/MVision/blob/master/CNN/HighPerformanceComputing/example/ncnn_%E6%96%B0%E5%BB%BA%E5%B1%82.md

NCNN 下载编译以及使用：

<https://github.com/Ewenwan/MVision/blob/master/CNN/HighPerformanceComputing/example/readme.md>

0x02. 运行流程解析

要了解一个深度学习框架，首先得搞清楚这个框架是如何通过读取一张图片然后获得的我们想要的输出结果，这个运行流程究竟是长什么样的？我们看一下 NCNN 官方 wiki 中提供一个示例代码：

```
#include <opencv2/core/core.hpp>
#include <opencv2/highgui/highgui.hpp>
```

```

#include "net.h"

int main()
{
    // opencv读取输入图片
    cv::Mat img = cv::imread("image.ppm", CV_LOAD_IMAGE_GRAYSCALE);
    int w = img.cols;
    int h = img.rows;

    // 减均值以及缩放操作，最后输入数据的值域为[-1,1]
    ncnn::Mat in = ncnn::Mat::from_pixels_resize(img.data,
    ncnn::Mat::PIXEL_GRAY, w, h, 60, 60);
    float mean[1] = { 128.f };
    float norm[1] = { 1/128.f };
    in.substract_mean_normalize(mean, norm);

    // 构建NCNN的net，并加载转换好的模型
    ncnn::Net net;
    net.load_param("model.param");
    net.load_model("model.bin");

    // 创建网络提取器，设置网络输入，线程数，light模式等等
    ncnn::Extractor ex = net.create_extractor();
    ex.set_light_mode(true);
    ex.set_num_threads(4);
    ex.input("data", in);
    // 调用extract接口，完成网络推理，获得输出结果
    ncnn::Mat feat;
    ex.extract("output", feat);

    return 0;
}

```

0x02.00 图像预处理 ncnn::Mat

可以看到 NCNN 对于我们给定的一个网络（首先转换为 NCNN 的 param 和 bin 文件）和输入，首先执行图像预处理，这是基于 **ncnn::Mat** 这个数据结构完成的。

其中，`from_pixels_resize()` 这个函数的作用是生成目标尺寸大小的网络输入 Mat，它的实现在

https://github.com/Tencent/ncnn/blob/b93775a27273618501a15a235355738cda102a38/src/mat_pixel.cpp#L2543。它的内部实际上是根据传入的输入图像的通道数完成

`resize_bilinear_c1/c2/c3/4` 即一通道 / 二通道 / 三通道 / 四通道 图像变形算法，可以看到使用的是双线性插值算法。这些操作的实现在

https://github.com/Tencent/ncnn/blob/master/src/mat_pixel_resize.cpp#L27。然后经过 `Resize` 之后，需要将像素图像转换成 `ncnn::Mat`。这里调用的是 `Mat::from_pixels()` 这个函数，它将我们 `Resize` 操作之后获得的像素图像数据（即 `float*` 数据）根据特定的输入类型赋值给 `ncnn::Mat`。

接下来，我们讲讲 `substract_mean_normalize()` 这个函数，它实现了减均值和归一化操作，它的实现在：<https://github.com/Tencent/ncnn/blob/master/src/mat.cpp#L34>。具体来说，

这个函数根据均值参数和归一化参数的有无分成这几种情况：

- 有均值参数
 - 创建 偏置层 `ncnn::create_layer(ncnn::LayerType::Bias);` 载入层参数 `op->load_param(pd);` 3 通道
 - 载入层权重数据 `op->load_model(ncnn::ModelBinFromMatArray(weights));` - 均值参数
 - 运行层 `op->forward_inplace(*this);`
- 有归一化参数
 - 创建 尺度层 `ncnn::create_layer(ncnn::LayerType::Scale);` 载入层参数 `op->load_param(pd);` 3 通道
 - 载入层权重数据 `op->load_model(ncnn::ModelBinFromMatArray(weights));` 尺度参数
 - 运行层 `op->forward_inplace(*this);`
- 有均值和归一化参数
 - 创建 尺度层 `ncnn::create_layer(ncnn::LayerType::Scale);` 载入层参数 `op->load_param(pd);` 3 通道
 - 载入层权重数据 `op->load_model(ncnn::ModelBinFromMatArray(weights));` - 均值参数 和 尺度参数
 - 运行层 `op->forward_inplace(*this);`

可以看到 NCNN 的均值和归一化操作，是直接利用了它的 Bias Layer 和 Scale Layer 来实现的，也就是说 NCNN 中的每个层都可以单独拿出来运行我们自己数据，更加方便我们~白嫖~。

0x02.01 模型解析 `ncnn::Net`

param 解析

完成了图像预处理之后，新增了一个 `ncnn::Net`，然后调用 `Net::load_param` 来载入网络参数文件 `*.proto`，这部分的实现在

<https://github.com/Tencent/ncnn/blob/master/src/net.cpp#L115>。在讲解这个函数在的过程之前，我们先来一起分析一下 NCNN 的 param 文件，举例如下：

```
7767517 # 文件头 魔数
75 83 # 层数量 输入输出blob数量
# 下面有75行
Input data 0 1 data 0=227 1=227 2=3
Convolution conv1 1 1 data conv1 0=64 1=3 2=1 3=2 4=0 5=1
6=1728
ReLU relu_conv1 1 1 conv1 conv1_relu_conv1 0=0.000000
Pooling pool1 1 1 conv1_relu_conv1 pool1 0=0 1=3 2=2 3=0
4=0
```

```

Convolution      fire2/squeeze1x1 1 1 pool1 fire2/squeeze1x1 0=16 1=1 2=1
3=1 4=0 5=1 6=1024
...
层类型          层名字    输入blob数量 输出blob数量  输入blob名字 输出blob名字 参
数字典

参数字典，每一层的意义不一样：
数据输入层 Input      data          0 1 data 0=227 1=227 2=3 图像宽
度×图像高度×通道数量
卷积层 Convolution ... 0=64 1=3 2=1 3=2 4=0 5=1
6=1728
0输出通道数 num_output(); 1卷积核尺寸 kernel_size(); 2空洞卷积参数
dilation(); 3卷积步长 stride();
4卷积填充pad_size(); 5卷积偏置有无bias_term(); 6卷积核参数数量
weight_blob.data_size();
C_OUT * C_in *
W_h * W_w = 64*3*3*3 = 1728
池化层 Pooling 0=0 1=3 2=2 3=0 4=0
0池化方式:最大值、均值、随机 1池化核大小 kernel_size();
2池化核步长 stride();
3池化核填充 pad(); 4是否为全局池化 global_pooling();
激活层 ReLU 0=0.000000 下限阈值 negative_slope();
ReLU6 0=0.000000 1=6.000000 上下限

综合示例：
0=1 1=2.5 -23303=2,2.0,3.0

数组关键字 : -23300
-(-23303) - 23300 = 3 表示该参数在参数数组中的index
后面的第一个参数表示数组元素数量, 2表示包含两个元素

```

然后官方的 wiki 中提供了所有网络层的详细参数设置，地址为：

<https://github.com/Tencent/mxnet/wiki/operation-param-weight-table>

了解了 Param 的基本含义之后，我们可以来看一下 `Net::load_param` 这个函数是在做什么了。

从函数实现，我们知道，首先会遍历 param 文件中的所有网络层，然后根据当前层的类型调用 `create_layer()/ net::create_custom_layer()` 来创建网络层，然后读取输入 Blobs 和输出 Blobs 和当前层绑定，再调用 `paramDict::load_param(fp)` 解析当前层的特定参数（参数字典），按照 `id=参数/参数数组` 来解析。最后，当前层调用 `layer->load_param(pd)` 载入解析得到的层特殊参数即获得当前层特有的参数。

核心代码解析如下：

```

// 参数读取 程序

// 读取字符串格式的 参数文件
int ParamDict::load_param(FILE* fp)
{
    clear();

    //      0=100 1=1.250000 -23303=5,0.1,0.2,0.4,0.8,1.0

    // parse each key=value pair

```

```

int id = 0;
while (fscanf(fp, "%d=", &id) == 1)// 读取 等号前面的 key=====
{
    bool is_array = id <= -23300;
    if (is_array)
    {
        id = -id - 23300;// 数组 关键字 -23300 得到该参数在参数数组中的 index
    }

// 是以 -23300 开头表示的数组=====
    if (is_array)
    {
        int len = 0;
        int nscan = fscanf(fp, "%d", &len);// 后面的第一个参数表示数组元素数量,
5表示包含两个元素
        if (nscan != 1)
        {
            fprintf(stderr, "ParamDict read array length fail\n");
            return -1;
        }

        params[id].v.create(len);

        for (int j = 0; j < len; j++)
        {
            char vstr[16];
            nscan = fscanf(fp, "%15[^\n]", vstr);//按格式解析字符串
=====
            if (nscan != 1)
            {
                fprintf(stderr, "ParamDict read array element fail\n");
                return -1;
            }

            bool is_float = vstr_is_float(vstr);// 检查该字段是否为 浮点数的字
字符串

            if (is_float)
            {
                float* ptr = params[id].v;
                nscan = sscanf(vstr, "%f", &ptr[j]);// 转换成浮点数后存入参数
字典中

            }
            else
            {
                int* ptr = params[id].v;
                nscan = sscanf(vstr, "%d", &ptr[j]);// 转换成 整数后 存入字典
中

            }
            if (nscan != 1)
            {
                fprintf(stderr, "ParamDict parse array element fail\n");
                return -1;
            }
        }
    }
}

// 普通关键字=====

```

```

else
{
    char vstr[16];
    int nscan = fscanf(fp, "%15s", vstr); // 获取等号后面的 字符串
    if (nscan != 1)
    {
        fprintf(stderr, "ParamDict read value fail\n");
        return -1;
    }

    bool is_float = vstr_is_float(vstr); // 判断是否为浮点数

    if (is_float)
        nscan = sscanf(vstr, "%f", &params[id].f); // 读入为浮点数
    else
        nscan = sscanf(vstr, "%d", &params[id].i); // 读入为整数
    if (nscan != 1)
    {
        fprintf(stderr, "ParamDict parse value fail\n");
        return -1;
    }
}

params[id].loaded = 1; // 设置该 参数以及载入
}

return 0;
}

// 读取 二进制格式的 参数文件=====
int ParamDict::load_param_bin(FILE* fp)
{
    clear();

    //      binary 0
    //      binary 100
    //      binary 1
    //      binary 1.250000
    //      binary 3 | array_bit
    //      binary 5
    //      binary 0.1
    //      binary 0.2
    //      binary 0.4
    //      binary 0.8
    //      binary 1.0
    //      binary -233(EOP)

    int id = 0;
    fread(&id, sizeof(int), 1, fp); // 读入一个整数长度的 index

    while (id != -233) // 结尾
    {
        bool is_array = id <= -23300;
        if (is_array)
        {
            id = -id - 23300; // 数组关键字对应的 index
        }
    }
}

```

```

// 是数组数据=====
    if (is_array)
    {
        int len = 0;
        fread(&len, sizeof(int), 1, fp); // 数组元素数量

        params[id].v.create(len);

        float* ptr = params[id].v;
        fread(ptr, sizeof(float), len, fp); // 按浮点数长度*数组长度 读取每一个数
组元素====
    }
// 是普通数据=====
    else
    {
        fread(&params[id].f, sizeof(float), 1, fp); // 按浮点数长度读取 该普通
字段对应的元素
    }

    params[id].loaded = 1;

    fread(&id, sizeof(int), 1, fp); // 读取 下一个 index
}

return 0;
}

```

bin 解析

解析完 param 文件，接下来需要对 bin 文件进行解析，这部分的实现在：

<https://github.com/Tencent/ncnn/blob/master/src/net.cpp#L672>。这里执行的主要的操作如下： - 创建 ModelBinFromStdio 对象 提供载入参数的接口函数

ModelBinFromStdio::load() 根据 权重数据开始的一个四字节数据类型参数

(float32/float16/int8 等) 和 指定的参数数量 读取数据到 Mat 并返回 Mat，这个函数的实现在

<https://github.com/Tencent/ncnn/blob/master/src/modelbin.cpp#L50>。 - 根据

load_param 获取到的网络层信息 遍历每一层 载入每一层的模型数据 layer->load_model() 每一

层特有函数。 - 部分层需要 根据层实际参数 调整运行流水线 layer->create_pipeline 例如卷积

层和全连接层 - 量化的网络需要融合 Net::fuse_network()

bin 文件的结构如下：

```

+-----+-----+-----+-----+-----+-----+
| weight1 | weight2 | weight3 | weight4 | ..... | weightN |
+-----+-----+-----+-----+-----+-----+
^           ^           ^           ^
0x0        0x80       0x140       0x1C0

```

所有权重数据连接起来，每个权重占 32bit。

权重数据 weight buffer

[flag] (optional 可选)

```
[raw data]
[padding] (optional 可选)

    flag : unsigned int, little-endian, indicating the weight storage type,
           0          => float32,
           0x01306B47 => float16,
           其它非0 => int8, 如果层实现显式强制存储类型, 则可以省略
    raw data : 原始权重数据、little endian、float32数据或float16数据或量化表和索引, 具体取决于存储类型标志
    padding : 32位对齐的填充空间, 如果已经对齐, 则可以省略。
```

感觉 bin 解析这部分了解一下就好, 如果感兴趣可以自己去看看源码。

0x02.03 网络运行 ncnn::Extractor

至此, 我们将网络的结构和权重信息都放到了 `ncnn::Net` 这个结构中, 接下来我们就可以新建网络提取器 `Extractor` `Net::create_extractor`, 它给我们提供了设置网络输入 (`Extractor::input`), 获取网络输出 (`Extractor::extract`), 设置网络运行线程参数 (`Extractor::set_num_threads`) 等接口。接下来, 我们只需要调用 `Extractor::extract` 运行网络 (`net`) 的前向传播函数 `net->forward_layer` 就可以获得最后的结果了。

另外, `ncnn::Extractor` 还可以设置一个轻模式省内存 即 `set_light_mode(true)`, 原理是 `net` 中每个 `layer` 都会产生 `blob`, 除了最后的结果和分支中间结果, 大部分 `blob` 都不值得保留, 开启轻模式可以在运算后自动回收, 省下内存。但需要注意的是, 一旦开启这个模式, 我们就不能获得中间层的特征值了, 因为中间层的内存在获得最终结果之前都被回收掉了。例如: 某网络结构为 **A -> B -> C**, 在轻模式下, 向 `ncnn` 索要 **C** 结果时, **A** 结果会在运算 **B** 时自动回收, 而 **B** 结果会在运算 **C** 时自动回收, 最后只保留 **C** 结果, 后面再需要 **C** 结果会直接获得, 满足大多数深度网络的使用方式。

最后, 我们需要明确一下, 我们刚才是先创建了 `ncnn::net`, 然后我们调用的 `ncnn::Extractor` 作为运算实例, 因此运算实例是不受 `net` 限制的。换句话说, 虽然我们只有一个 `net`, 但我们可以开多个 `ncnn::Extractor`, 这些实例都是单独完成特定网络的推理, 互不影响。

这样我们就大致了解了 NCNN 的运行流程了, 更多的细节可以关注 NCNN 源码。

0x03. NCNN 源码目录分析

这一节, 我们分析一下 NCNN 源码目录以便更好的理解整个工程。src 的目录结构如下:

- /src 目录:
 - ./src/layer 下是所有的 layer 定义代码
 - ./src/layer/arm 是 arm 下的计算加速的 layer
 - ./src/layer/x86 是 x86 下的计算加速的 layer。

- ./src/layer/mips 是 mips 下的计算加速的 layer。
- ./src/layer/.h + ./src/layer/.cpp 是各种 layer 的基础实现，无加速。
- 目录顶层下是一些基础代码，如宏定义，平台检测，mat 数据结构，layer 定义，blob 定义，net 定义等。
- platform.h.in 平台检测
- benchmark.h + benchmark.cpp 测试各个模型的执行速度
- allocator.h + allocator.cpp 内存池管理，内存对齐
- paramdict.h + paramdict.cpp 层参数解析 读取二进制格式、字符串格式、密文格式的
参数文件
- opencv.h opencv.cpp opencv 风格的数据结构 的 mini 实现，包含大小结构体 Size，
矩阵框结构体 Rect_ 交集 并集运算符重载，点结构体 Point_，矩阵结构体 Mat 深拷贝
浅拷贝 获取指定矩形框中的 roi 读取图像 写图像 双线性插值算法改变大小等等
- mat.h mat.cpp 三维矩阵数据结构，在层间传播的就是 Mat 数据，Blob 数据是工具人，
另外包含 subtract_mean_normalize(), 去均值并归一化；half2float(), float16 的
data 转换成 float32 的 data; copy_make_border(), 矩阵周围填充；
resize_bilinear_image(), 双线性插值等函数。
- net.h net.cpp ncnn 框架接口，包含注册 用户定义的新层
Net::register_custom_layer(); 网络载入 模型参数 Net::load_param(); 载入 模型权重
Net::load_model(); 网络 blob 输入 Net::input(); 网络前向传播 Net::forward_layer();
被 Extractor::extract() 执行；创建网络模型提取器 Net::create_extractor(); 模型提取器
提取某一层输出 Extractor::extract() 等函数。
- ...

源码目录除了这些还有很多文件，介于篇幅原因就不再枚举了，感兴趣的可以自行查看源码。由于我只对 x86 和 arm 端的指令集加速熟悉一些，所以这里再枚举一下 `src/layers` 下面的 NCNN 支持的层的目录：

```

|— absval.cpp                                // 绝对值层
|— absval.h
|— argmax.cpp                                // 最大值层
|— argmax.h
|— arm ===== arm平台下的层
|   |— absval_arm.cpp                        // 绝对值层
|   |— absval_arm.h
|   |— batchnorm_arm.cpp                    // 批归一化 去均值除方差
|   |— batchnorm_arm.h
|   |— bias_arm.cpp                         // 偏置
|   |— bias_arm.h
|   |— convolution_1x1.h                    // 1*1 float32 卷积
|   |— convolution_1x1_int8.h              // 1*1 int8 卷积
|   |— convolution_2x2.h                    // 2*2 float32 卷积
|   |— convolution_3x3.h                    // 3*3 float32 卷积
|   |— convolution_3x3_int8.h              // 3*3 int8 卷积
|   |— convolution_4x4.h                    // 4*4 float32 卷积

```

```

| |— convolution_5x5.h // 5*5 float32 卷积
| |— convolution_7x7.h // 7*7 float32 卷积
| |— convolution_arm.cpp // 卷积层
| |— convolution_arm.h
| |— convolutiondepthwise_3x3.h // 3*3 逐通道 float32 卷积
| |— convolutiondepthwise_3x3_int8.h // 3*3 逐通道 int8 卷积
| |— convolutiondepthwise_arm.cpp // 逐通道卷积
| |— convolutiondepthwise_arm.h
| |— deconvolution_3x3.h // 3*3 反卷积
| |— deconvolution_4x4.h // 4*4 反卷积
| |— deconvolution_arm.cpp // 反卷积
| |— deconvolution_arm.h
| |— deconvolutiondepthwise_arm.cpp // 反逐通道卷积
| |— deconvolutiondepthwise_arm.h
| |— dequantize_arm.cpp // 反量化
| |— dequantize_arm.h
| |— eltwise_arm.cpp // 逐元素操作, product(点乘), sum(相加减)
和 max(取大值)
| |— eltwise_arm.h
| |— innerproduct_arm.cpp // 即 fully_connected (fc)layer, 全连接
层
| |— innerproduct_arm.h
| |— lrn_arm.cpp // Local Response Normalization, 即局部
响应归一化层
| |— lrn_arm.h
| |— neon_mathfun.h // neon 数学函数库
| |— pooling_2x2.h // 2*2 池化层
| |— pooling_3x3.h // 3*3 池化层
| |— pooling_arm.cpp // 池化层
| |— pooling_arm.h
| |— prelu_arm.cpp // (a*x,x) 前置relu激活层
| |— prelu_arm.h
| |— quantize_arm.cpp // 量化层
| |— quantize_arm.h
| |— relu_arm.cpp // relu 层 (0,x)
| |— relu_arm.h
| |— scale_arm.cpp // BN层后的 平移和缩放层 scale
| |— scale_arm.h
| |— sigmoid_arm.cpp // sigmoid 负指数倒数归一化 激活层  $1/(1 + e^{(-z)})$ 
| |— sigmoid_arm.h
| |— softmax_arm.cpp // softmax 指数求和归一化 激活层  $e^{(z)}/\sum(e^{(z)})$ 
/ sum(e^(zi))
| |— softmax_arm.h
|
|===== 普通平台 待优化=====
|— batchnorm.cpp // 批归一化 去均值除方差
|— batchnorm.h
|— bias.cpp // 偏置
|— bias.h
|— binaryop.cpp // 二元操作: add, sub, div, mul, mod等
|— binaryop.h
|— bnll.cpp // binomial normal log likelihood的简称
f(x)=log(1 + exp(x)) 激活层
|— bnll.h
|— clip.cpp // 截断=====

```

```

└─ clip.h
└─ concat.cpp // 通道叠加
└─ concat.h
└─ convolution.cpp // 普通卷积层
└─ convolutiondepthwise.cpp // 逐通道卷积
└─ convolutiondepthwise.h
└─ convolution.h
└─ crop.cpp // 剪裁层
└─ crop.h
└─ deconvolution.cpp // 反卷积
└─ deconvolutiondepthwise.cpp // 反逐通道卷积
└─ deconvolutiondepthwise.h
└─ deconvolution.h
└─ dequantize.cpp // 反量化
└─ dequantize.h
└─ detectionoutput.cpp // ssd 的检测输出层
=====
└─ detectionoutput.h
└─ dropout.cpp // 随机失活层 在训练时由于舍弃了一些神经元,因此在测试时需
要在激励的结果中乘上因子p进行缩放.
└─ dropout.h
└─ elwise.cpp // 逐元素操作, product(点乘), sum(相加减) 和 max(取
大值)
└─ elwise.h
└─ elu.cpp // 指数线性单元relu激活层 Prelu : (a*x, x) ---->
Erelu : (a*(e^x - 1), x)
└─ elu.h
└─ embed.cpp // 嵌入层, 用在网络的开始层将你的输入转换成向量
└─ embed.h
└─ expanddims.cpp // 增加维度
└─ expanddims.h
└─ exp.cpp // 指数映射
└─ exp.h
└─ flatten.cpp // 摊平层
└─ flatten.h
└─ innerproduct.cpp // 全连接层
└─ innerproduct.h
└─ input.cpp // 数据输入层
└─ input.h
└─ instancenorm.cpp // 单样本 标准化 规范化
└─ instancenorm.h
└─ interp.cpp // 插值层 上下采样等
└─ interp.h
└─ log.cpp // 对数层
└─ log.h
└─ lrn.cpp // Local Response Normalization, 即局部响应归一化层
└─ lrn.h // 对局部神经元的活动创建竞争机制, 使得其中响应比较大的值
变得相对更大,
// 并抑制其他反馈较小的神经元, 增强了模型的泛化能力
└─ lstm.cpp // lstm 长短词记忆层
└─ lstm.h // lstm 长短词记忆层
└─ memorydata.cpp // 内存数据层
└─ memorydata.h
└─ mvn.cpp
└─ mvn.h
└─ normalize.cpp // 归一化
└─ normalize.h

```

```

└─ padding.cpp // 填充, 警戒线
└─ padding.h
└─ permute.cpp // ssd 特有层 交换通道顺序 [bantch_num, channels,
h, w] ----> [bantch_num, h, w, channels]]=====
└─ permute.h
└─ pooling.cpp // 池化层
└─ pooling.h
└─ power.cpp // 平移缩放乘方 : (shift + scale * x) ^ power
└─ power.h
└─ prelu.cpp // Prelu (a*x,x)
└─ prelu.h
└─ priorbox.cpp // ssd 独有的层 建议框生成层 L1 loss 拟合
=====
└─ priorbox.h
└─ proposal.cpp // faster rcnn 独有的层 建议框生成, 将rpn网络的输出转
换成建议框=====
└─ proposal.h
└─ quantize.cpp // 量化层
└─ quantize.h
└─ reduction.cpp // 将输入的特征图按照给定的维度进行求和或求平均
└─ reduction.h
└─ relu.cpp // relu 激活层: (0,x)
└─ relu.h
└─ reorg.cpp // yolov2 独有的层, 一拆四层, 一个大矩阵, 下采样到四个
小矩阵=====
└─ reorg.h
└─ reshape.cpp // 变形层: 在不改变数据的情况下, 改变输入的维度
└─ reshape.h
└─ rnn.cpp // rnn 循环神经网络
└─ rnn.h
└─ roipooling.cpp // faster Rcnn 独有的层, ROI池化层: 输入m*n 均匀划
分成 a*b个格子后池化, 得到固定长度的特征向量 =====
└─ roipooling.h
└─ scale.cpp // bn 层之后的 平移缩放层
└─ scale.h
└─ shufflechannel.cpp // ShuffleNet 独有的层, 通道打乱, 通道混合层
=====
└─ shufflechannel.h
└─ sigmoid.cpp // 负指数倒数归一化层  $1/(1 + e^{(-zi)})$ 
└─ sigmoid.h
└─ slice.cpp // concat的反向操作, 通道分开层, 适用于多任务网络
└─ slice.h
└─ softmax.cpp // 指数求和归一化层  $e^{(zi)} / \sum(e^{(zi)})$ 
└─ softmax.h
└─ split.cpp // 将blob复制几份, 分别给不同的layer, 这些上层layer共享
这个blob。
└─ split.h
└─ spp.cpp // 空间金字塔池化层 1+4+16=21 SPP-NET 独有
=====
└─ spp.h
└─ squeeze.cpp // squeezeNet独有层, Fire Module, 一层conv层变成两
层: squeeze层+expand层, 1*1卷积----> 1*1 + 3*3=====
└─ squeeze.h
└─ tanh.cpp // 双曲正切激活函数  $(e^{(zi)} - e^{(-zi)}) / (e^{(zi)} + e^{(-zi)})$ 
└─ tanh.h
└─ threshold.cpp // 阈值函数层

```

```

├─ threshold.h
├─ tile.cpp                // 将blob的某个维度，扩大n倍。比如原来是1234，扩大两倍变成11223344。
├─ tile.h
├─ unaryop.cpp            // 一元操作：abs, sqrt, exp, sin, cos, conj (共轭) 等
├─ unaryop.h
├─
├─ =====x86下特殊的优化层=====
├─ x86
│   ├── avx_mathfun.h      // x86 数学函数
│   ├── convolution_1x1.h  // 1*1 float32 卷积
│   ├── convolution_1x1_int8.h // 1*1 int8 卷积
│   ├── convolution_3x3.h  // 3*3 float32 卷积
│   ├── convolution_3x3_int8.h // 3*3 int8 卷积
│   ├── convolution_5x5.h  // 5*5 float32 卷积
│   ├── convolutiondepthwise_3x3.h // 3*3 float32 逐通道卷积
│   ├── convolutiondepthwise_3x3_int8.h // 3*3 int8 逐通道卷积
│   ├── convolutiondepthwise_x86.cpp // 逐通道卷积
│   ├── convolutiondepthwise_x86.h
│   ├── convolution_x86.cpp // 卷积
│   ├── convolution_x86.h
│   └─ sse_mathfun.h      // sse优化 数学函数
├─ yolodetectionoutput.cpp // yolo-v2 目标检测输出层
├─ =====
└─ yolodetectionoutput.h

```

当然还有一些支持的层没有列举到，具体以源码为准。

0x04. NCNN 是如何加速的？

之所以要单独列出这部分，是因为 NCNN 作为一个前向推理框架，推理速度肯定是尤其重要的。所以这一节我就来科普一下 NCNN 为了提升网络的运行速度做了哪些关键优化。我们需要明确一点，当代 CNN 的计算量主要集中在卷积操作上，只要卷积层的速度优化到位，那么整个网络的运行速度就能获得极大提升。所以，我们这里先以卷积层为例来讲讲 NCNN 是如何优化的。

在讲解之前，先贴出我前面很长一段时间学习的一些优化策略和复现相关的文章链接，因为这些思路至少一半来自于 NCNN，所以先把链接汇总在这里，供需要的小伙伴获取。

- [一份朴实无华的移动端盒子滤波算法优化笔记](#)
- [基于 NCNN 的 3x3 可分离卷积再思考盒子滤波](#)
- [详解 Im2Col+Pack+Sgemm 策略更好的优化卷积运算](#)
- [详解卷积中的 Winograd 加速算法](#)
- [道阻且长_再探矩阵乘法优化](#)

NCNN 中对卷积的加速过程 (以 Arm 侧为例) 在我看来有：

- 无优化
- 即用即取 + 共用行
- Im2Col+GEMM
- WinoGrad
- SIMD
- 内联汇编
- 针对特定架构如 A53 和 A55 提供更好的指令排布方式，不断提高硬件利用率

后面又加入了 Pack 策略，更好的改善访存，进一步提升速度。

不得不说，NCNN 的底层优化做得还是比较细致的，所以大家一定要去~白嫖~啊。这里列举的是 Arm 的优化策略，如果是 x86 或者其它平台以实际代码为准。

下面贴一个带注释的 ARM neon 优化绝对值层的例子作为结束吧，首先绝对值层的普通 C++ 版本如下：

```
// 绝对值层特性：单输入，单输出，可直接对输入进行修改
int AbsVal::forward_inplace(Mat& bottom_top_blob, const Option& opt) const
{
    int w = bottom_top_blob.w;    // 矩阵宽度
    int h = bottom_top_blob.h;    // 矩阵高度
    int channels = bottom_top_blob.c; // 通道数
    int size = w * h; // 一个通道的元素数量

    #pragma omp parallel for num_threads(opt.num_threads) // openmp 并行
    for (int q=0; q<channels; q++) // 每个 通道
    {
        float* ptr = bottom_top_blob.channel(q); // 当前通道数据的起始指针

        for (int i=0; i<size; i++) // 遍历每个值
        {
            if (ptr[i] < 0)
                ptr[i] = -ptr[i]; // 小于零取相反数，大于零保持原样
            // ptr[i] = ptr[i] > 0 ? ptr[i] : -ptr[i];
        }
    }

    return 0;
}
```

ARM neon 优化版本如下：

```
// arm 内联汇编
// asm(
// 代码列表
// : 输出运算符列表      "r" 表示同用寄存器  "m" 表示内存地址 "I" 立即数
// : 输入运算符列表      "=r" 修饰符 = 表示只写，无修饰符表示只读，+修饰符表示可读可写，
// &修饰符表示只作为输出
// : 被更改资源列表
// );
```

```
// __asm__ __volatile__();  
// __volatile__或volatile 是可选的，假如用了它，则是向GCC 声明不答应对该内联汇编优化，  
// 否则当 使用了优化选项(-O)进行编译时，GCC 将会根据自己的判定决定是否将这个内联汇编表达式中的  
指令优化掉。
```

```
// 换行符和制表符的使用可以使得指令列表看起来变得美观。
```

```
int AbsVal_arm::forward_inplace(Mat& bottom_top_blob, const Option& opt) const  
{
```

```
    int w = bottom_top_blob.w;    // 矩阵宽度  
    int h = bottom_top_blob.h;    // 矩阵高度  
    int channels = bottom_top_blob.c; // 通道数  
    int size = w * h; // 一个通道的元素数量
```

```
    #pragma omp parallel for num_threads(opt.num_threads)
```

```
    for (int q=0; q<channels; q++)  
    {
```

```
        float* ptr = bottom_top_blob.channel(q);
```

```
    #if __ARM_NEON
```

```
        int nn = size >> 2; // 128位的寄存器，一次可以操作 4个float, 剩余不够4个的，最后  
        后面直接c语言执行
```

```
        int remain = size - (nn << 2); // 4*32 =128字节对其后 剩余的 float32个数，  
        剩余不够4个的数量
```

```
    #else
```

```
        int remain = size;
```

```
    #endif // __ARM_NEON
```

```
    /*
```

```
    从内存中载入：
```

```
    v7:
```

```
        带了前缀v的就是v7 32bit指令的标志；
```

```
        ld1表示是顺序读取，还可以取ld2就是跳一个读取，ld3、ld4就是跳3、4个位置读取，这在RGB分解  
        的时候贼方便；
```

```
        后缀是f32表示单精度浮点，还可以是s32、s16表示有符号的32、16位整型值。
```

```
        这里Q寄存器是用q表示，q5对应d10、d11可以分开单独访问（注：v8就没这么方便了。）
```

```
        大括号里面最多只有两个Q寄存器。
```

```
        "vld1.f32 {q10}, [%3]!          \n"
```

```
        "vld1.s16 {q0, q1}, [%2]!      \n"
```

```
    v8:
```

```
        ARMv8 (64位cpu) NEON寄存器 用 v来表示 v1.8b v2.8h v3.4s v4.2d
```

```
        后缀为8b/16b/4h/8h/2s/4s/2d)
```

```
        大括号内最多支持4个V寄存器；
```

```
        "ld1 {v0.4s, v1.4s, v2.4s, v3.4s}, [%2], #64 \n" // 4s表示float32
```

```
        "ld1 {v0.8h, v1.8h}, [%2], #32 \n"
```

```
        "ld1 {v0.4h, v1.4h}, [%2], #32 \n" // 4h 表示int16
```

```
    */
```

```
    #if __ARM_NEON
```

```
    #if __aarch64__
```

```
    // ARMv8-A 是首款64 位架构的ARM 处理器，是移动手机端使用的CPU
```

```
        if (nn > 0)
```

```
        {
```

```

asm volatile(
    "0:                                \n" // 0: 作为标志, 局部标签
    "prfm      pldl1keep, [%1, #128] \n" // 预取 128个字节 4*32 =
128
    "ld1       {v0.4s}, [%1]          \n" // 载入 ptr 指针对应的值, 连
续4个
    "fabs      v0.4s, v0.4s           \n" // ptr 指针对应的值 连续4个,
使用fabs函数 进行绝对值操作 4s表示浮点数
    "subs      %w0, %w0, #1           \n" // %0 引用 参数 nn 操作次数
每次 -1 #1表示1
    "st1       {v0.4s}, [%1], #16     \n" // %1 引用 参数 ptr 指针 向
前移动 4*4=16字节
    "bne       0b                     \n" // 如果非0, 则向后跳转到 0标志
处执行

    : "=r"(nn), // %0 操作次数
      "=r"(ptr) // %1
    : "0"(nn), // %0 引用 参数 nn
      "1"(ptr) // %1 引用 参数 ptr
    : "cc", "memory", "v0" /* 可能变化的部分 memory内存可能变化*/
);
}

#else
// 32位 架构处理器=====
if (nn > 0)
{
asm volatile(
    "0:                                \n" // 0: 作为标志, 局部标签
    "vld1.f32  {d0-d1}, [%1]          \n" // 载入 ptr处的值 q0寄存器 =
d0 = d1
    "vabs.f32  q0, q0                 \n" // abs 绝对值运算
    "subs      %0, #1                 \n" // %0 引用 参数 nn 操作次数每次
-1 #1表示1
    "vst1.f32  {d0-d1}, [%1]!         \n" // %1 引用 参数 ptr 指针 向前移
动 4*4=16字节
    "bne       0b                     \n" // 如果非0, 则向后跳转到 0标志处
执行

    : "=r"(nn), // %0
      "=r"(ptr) // %1
    : "0"(nn),
      "1"(ptr)
    : "cc", "memory", "q0" /* 可能变化的部分 memory内存可
能变化*/
);
}
#endif // __aarch64__
#endif // __ARM_NEON
for (; remain>0; remain--) // 剩余不够4个的直接c语言执行
{
    *ptr = *ptr > 0 ? *ptr : -*ptr;

    ptr++;
}
}

return 0;
}

```


0x05. 结语

介绍到这里就要结束了，这篇文章只是以我自己的视角看了一遍 NCNN，如果有什么错误或者笔误欢迎评论区指出。在 NCNN 之后各家厂商纷纷推出了自己的开源前向推理框架，例如 MNN，OpenAILab 的 Tengine，阿里的 tengine，旷视的 MegEngine，华为 Bolt 等等，希望各个 CVer 都能多多支持国产端侧推理框架。

0x06. 友情链接

- <https://github.com/Tencent/ncnn>
- <https://github.com/MegEngine/MegEngine>
- <https://github.com/alibaba/tengine>
- <https://github.com/OAID/Tengine>
- <https://github.com/alibaba/MNN>
- <https://github.com/Ewenwan/MVision>

欢迎关注 GiantPandaCV, 在这里你将看到独家的深度学习分享，坚持原创，每天分享我们学习到的新鲜知识。(•̀ω•́)✧

有对文章相关的问题，或者想要加入交流群，欢迎添加 BBuf 微信：



<https://blog.csdn.net/GiantPandaCV/>

为了方便读者获取资料以及我们公众号的作者发布一些 Github 工程的更新，我们成立了一个 QQ 群，二维码如下，感兴趣可以加入。



群名称:GiantPandaCV 交流群
群 号:522799362

Powered by

撰写评论

发布

还没有评论，快来抢沙发吧！

당신이 좋아할 만한 콘텐츠

Taboola 후원링크

眼袋看起來老10歲！只是塗萬寧買的「這個」

japonelifeskeem-hk.com

指甲上長有「紋」？八成是3種病找上門，早點自查

www.rebang360.com

腸道裡的脂肪瞬間化為烏有～！萬寧爆賣的益生菌，會不會瘦過頭？

熱門話題

42歲男性只吃數個月就判若兩人！用萬寧賣的「這個」超有效

熱門話題 | 好速纖 HELASLIM

只是塗萬寧賣到翻的「煙酰胺」就跟斑點分手了

japonelifeskeem-hk.com

人老了，長壽重點不在於運動，也不是喝水，過55歲後要謹記4點

Rebang 360

Hong Kong 的單身女性概況 - 查看全部

網上約會 | 搜尋廣告

立即搜索

Hong Kong 一日內全口植牙：老年人的價格可能會讓您感到驚訝

Local Plan

本文总阅读量1022次